API Testing

A cura di:

Pasquale De Nisi mat.252271
Francesco Magnone mat.269888

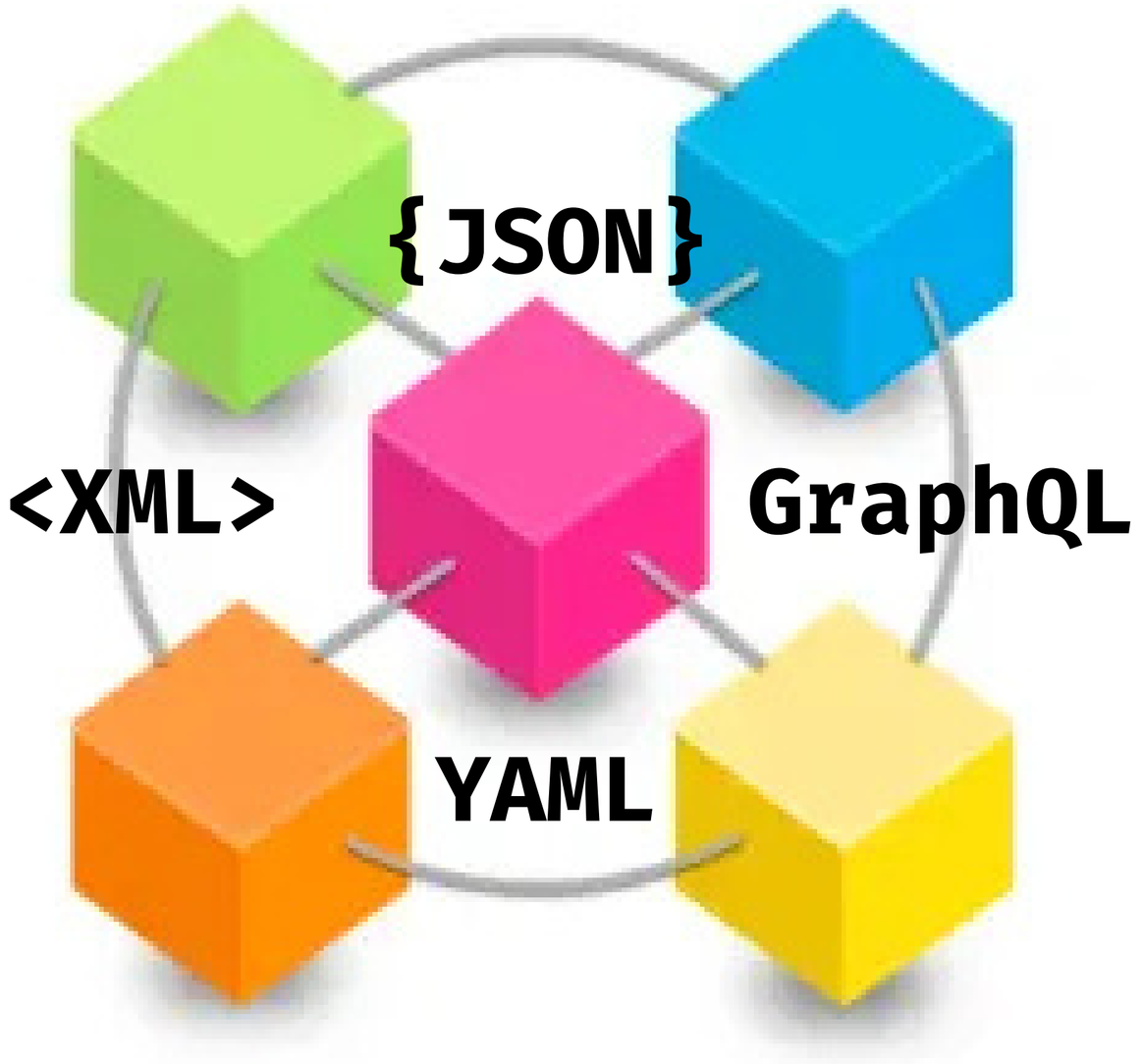
API

Application Programming Interface

Enhances efficient data exchange and sharing among different microsevices in modern architectures

Pros:

- Scalability
- Flexibility
- Fast deployments
- Responsability separation



Docs

Often public, expecially if the api is meant to be used by external developers.

Can be:

- human readable
- machine readable (xml/json)

Docs can be exploited to find logic flaws in the implementation.

Common path used:

- /api
- /swagger/index.html
- /openapi.json

Playing factors

Unexpected behaviours can be triggered by playing with

- data types
- optional and mandatory parameters
- http methods, if left open expand attack surface
- media format
- authentication
- rate limits

Hidden endpoints and parameters

Naming conventions, default values and industry terms can be used to discover hidden endpoints.

For example those can be added to the endpoint:

- /update
- /delete
- /add

Undocumented parameters supported by the API can be exploited to change the application's behaviour. These parameters can be discovered by manually examining objects returned by the API.

Mass assignment

Automatically bind endpoint parameters to object fields makes the application support undesired hidden parameters.

Pericoloso:
accetta TUTTI i
campi dal JSON

```python
@app.route('/api/users/<id>', methods=['PUT'])
def update_user(id):
    user = User.query.get(id)
    for key, value in request.json.items():
        setattr(user, key, value)
    db.session.commit()
```


```python
@app.route('/api/users/<id>', methods=['PUT'])
def update_user(id):
    user = User.query.get(id)
    allowed = ['name', 'email', 'bio']
    for key in allowed:
        if key in request.json:
            setattr(user, key, request.json[key])
    db.session.commit()
```

isAdmin viene ignorato
anche se presente
nella request

Server side parameter pollution

User input can be exploited to inject parameters in internal APIs not directly exposed on the internet by the application.

Wrong implementations can allow:

- Access to unauthorized data
- parameter overwriting
- unexpected behaviours

```
GET
/userSearch?
name=peter%26name=carlos
&back=/home
```

PHP parses the last parameter only, resulting in the user search for carlos overwriting the original peter

Server side parameter pollution

An attacker may be able to manipulate server-side URL path parameters to exploit the API. To test for this vulnerability, add path traversal sequences to modify parameters

```
GET /edit_profile.php?name=peter
```

This results in the following server-side request:

```
GET /api/private/users/peter
```

submit URL-encoded peter/../admin as the value of the name parameter

```
GET /edit_profile.php?name=peter%2f..%2fadmin
```

This may result in the following server-side request:

```
GET /api/private/users/peter/../admin
```

If the server-side client or back-end API normalize this path, it may be resolved to /api/private/users/admin.

Server side parameter pollution

An attacker may inject unexpected structured data into user inputs to exploit vulnerabilities in the server's processing of structured data formats, such as a JSON or XML.

Users can edit their profile. Changes are applied with a request to a server-side API. When a name is edited, the browser makes the following request:

```
POST /myaccount name=peter
```

This results in the following server-siderequest:

```
PATCH /users/7312/update {"name":"peter"}
```

Add access_level parameter

```
POST /myaccount name=peter",
"access_level":"administrator
```

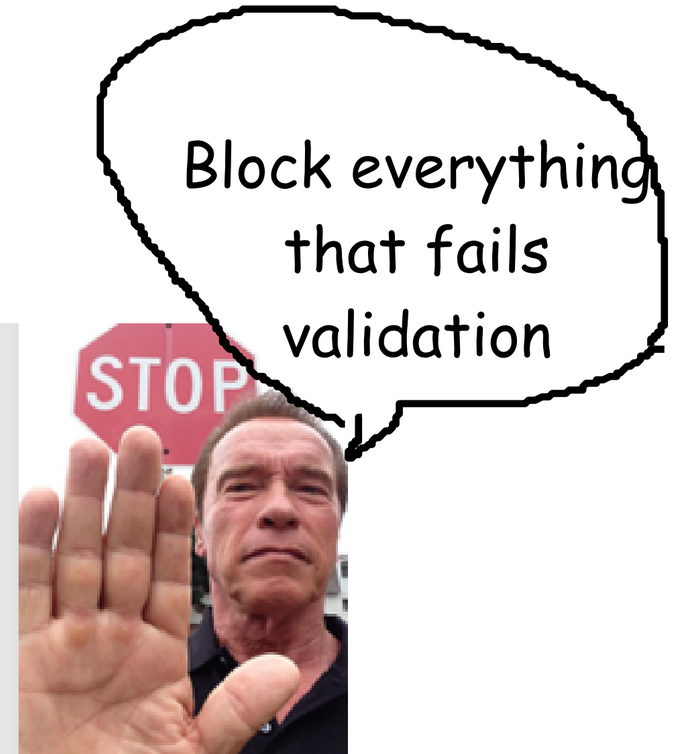
If input is not validated, this server-side request can be made:

```
PATCH /users/7312/update
{name="peter","access_level":"administrator"}
```

Peter has now administrator access.

Prevention

- Hide documentation if not meant to be public
- Keep docs updated to make the attack surface clear to legitimate developers
- Use allow lists for
  - allowed methods
  - editable field to prevent mass assignments
  - characters that don’t need encoding to prevent SSPP
- Validate content type of both requests and responses
- Use generic error messages
- Use security measures in every phase and not just in production
- Make sure all not white listed user input is encoded before it's included in a server-side request.
- Make sure that all input adheres to the expected format and structure.



Questions